Architecture proposal · PER #60254 + PER #60145

Composer assistance layer for Agent Desktop

Requesting tenant 2P / GOSI Found in 5.35 Status Draft for PM review

Date September 15, 2026

1 High-level overview

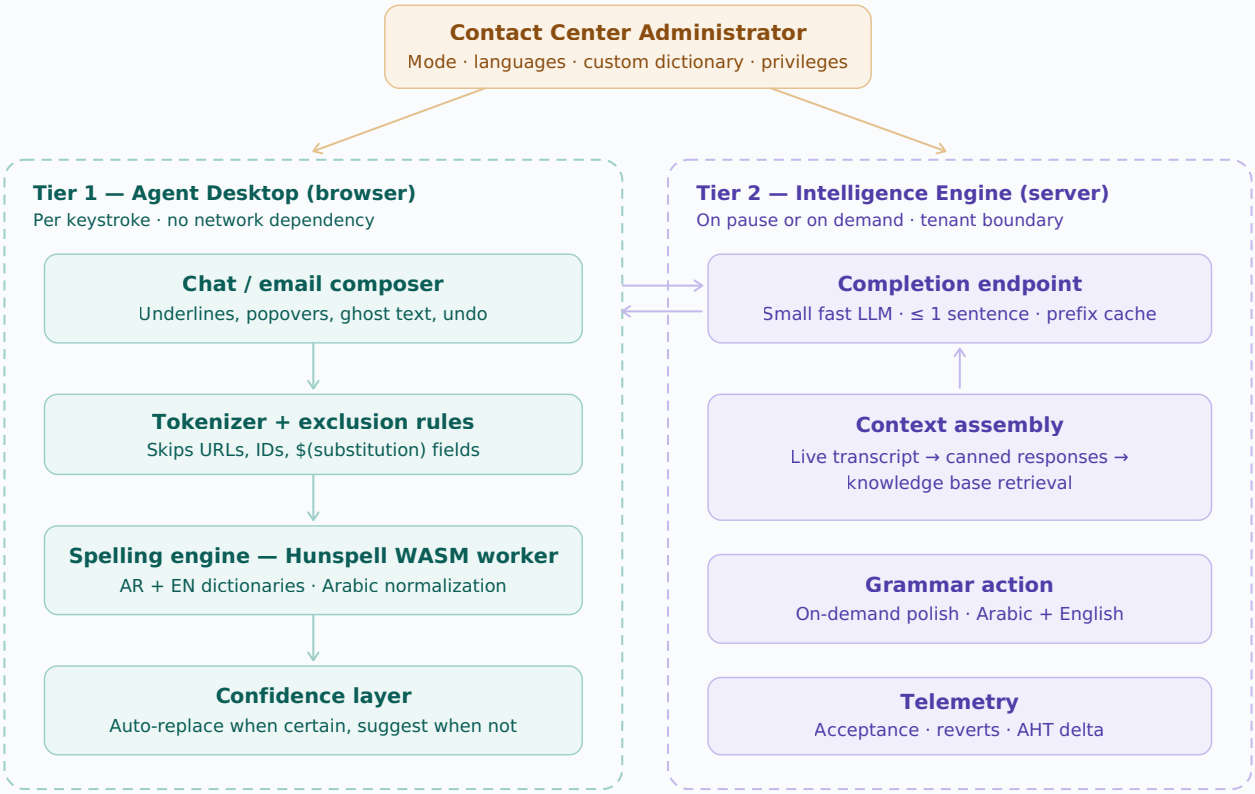
Two open enhancement requests from GOSI — auto-correct (#60254) and Gmail-style inline completion (#60145) — are best delivered as one platform capability: a composer assistance layer inside Agent Desktop that watches the agent's typing in the chat and email editors and helps in three escalating ways.

The layer provides three services behind a single integration point. Spell checking flags misspelled Arabic and English words as the agent types. An auto-correct decision layer silently fixes high-confidence slips and offers suggestions for ambiguous ones. Inline completion predicts the rest of the agent's sentence as dismissible ghost text, grounded in the live conversation, the service's canned responses, and the tenant knowledge base.

The defining constraint comes from the customer: GOSI's InfoSec policy prohibits browser extensions, so everything must be native to the product. That splits the architecture into two tiers with very different latency and infrastructure profiles. Everything per-keystroke runs entirely in the agent's browser with no network dependency; everything model-driven runs server-side in the Intelligence Engine, invoked only on typing pauses or explicit agent action. A single administration surface in Contact Center Administrator governs both tiers, satisfying GOSI's requirement that the feature be enforceable globally or per team by an administrator while agents retain the choice to accept or ignore individual suggestions.

GOSI's stated objectives — lower average handling time, more consistent customer communication, fewer spelling mistakes, and faster ramp-up for new agents — map directly onto these services, and each is measurable: acceptance rate, revert rate, and AHT delta are logged from day one.

2 Architecture at a glance



The composer talks to Tier 1 continuously and to Tier 2 only on typing pauses (completion) or explicit action (grammar polish). Admin configuration governs both tiers.

3 Components

Composer integration layer `tier 1`

A single hook into the Agent Desktop 2.0 chat and email editors that all three services share. It observes text changes on a debounce, maintains the token map, and renders every visual affordance without mutating the editable DOM: misspelling underlines are drawn with the CSS Custom Highlight API, and completion ghost text lives in a decoration layer after the caret, so nothing the layer draws can ever leak into a sent message or corrupt the agent's undo history. Accepted suggestions and auto-replacements are applied through the editor's own transaction path, keeping undo coherent — a single undo reverts any automatic change and adds the original word to a per-session ignore list. The layer sets `spellcheck="false"` on managed fields so agents never see two disagreeing checkers.

Tokenizer and exclusion rules tier 1

Words are checked only once they are settled — after the agent types a boundary character — never while the caret is inside them. Tokens matching URLs, email addresses, case and reference numbers, tokens containing digits, and Bright Pattern substitution fields such as `$(user.firstName)` are excluded from all processing. Script detection per token (Arabic vs Latin Unicode ranges) routes each word to the right dictionary, so the mixed Arabic-and-English messages typical of GOSI's traffic are handled naturally within one sentence.

Spelling engine tier 1

Hunspell compiled to WebAssembly, running in a web worker so dictionary loads and suggestion generation never block typing. Hunspell is elected under its MPL 1.1 license option (it is tri-licensed; no GNU license is taken), vendored at a pinned tag and compiled in CI with a minimal binding surface. Dictionaries are the standard `.aff` / `.dic` pairs: SCOWL-derived English (permissive license) and an Arabic pair whose licensing is an open item — AyaSpell must be cleared by legal, with a commercial dictionary or a clean-room wordlist built on MIT-licensed CAMeL Tools as fallbacks. Arabic tokens are checked raw first, then with normalization (diacritics stripped, tatweel removed, alef variants folded) so valid diacritized forms are never flagged. Dictionaries and the compiled module are cached client-side and versioned by tenant configuration, so an admin dictionary update propagates on next login. Engine construction happens at login, not first keystroke; the composer relies on browser-native spellcheck until the engine signals ready.

Auto-correct confidence layer tier 1

The policy brain that satisfies GOSI's requested behavior: automatic correction of clear mistakes, suggestion of uncertain ones. A misspelled word is auto-replaced only when the top Hunspell candidate is a single-edit slip on a word of four or more characters, preserves the first letter, and clearly dominates alternatives on a per-language word-frequency ranking; anything less confident gets a wavy underline and a suggestion popover. Auto-replacements flash briefly and are one-click revertible. Custom dictionary entries pushed from the admin configuration are injected into the engine at load, which is what prevents the layer from mangling product names, GOSI terminology, and Arabic transliterations.

Inline completion service tier 2

The smart compose half, per #60145's clarified requirements: suggestions appear inside the same editor as ghost text, in the Gmail Smart Compose interaction pattern — Tab accepts, typing through dismisses. When the agent pauses for ~300–500 ms mid-sentence, the client sends the partial sentence to a completion endpoint in the Intelligence Engine backed by a small, fast multilingual model constrained to at most one sentence at low temperature. The end-to-end latency budget is 200–400 ms; in-flight requests are cancelled on the next keystroke, triggers are suppressed for very short prefixes, and the

conversation context is prefix-cached server-side since only the partial sentence changes between consecutive triggers. Context is assembled in cheap-to-expensive order: the live interaction transcript always; the service's canned responses and approved phrasing via embedding retrieval, which is what steers completions toward consistent, approved wording; and knowledge base passages only when the customer's last message retrieves with high confidence. This makes completion a complement to canned responses and Agent Assist — it converges on them rather than competing with them — exactly as the customer requested.

Grammar action `tier 2`

From #60254's clarification, grammar is secondary to spelling and is wanted for both Arabic and English. Rule-based grammar engines have effectively no Arabic coverage, so grammar runs as an on-demand action against the same Intelligence Engine tier — a polish control in the composer, matching the button-style interaction the customer already knows from Konnect Insights — rather than per keystroke. This keeps all message text inside the Bright Pattern tenant boundary, the same compliance posture that ruled out extensions.

Administration and telemetry `config`

One settings surface in Contact Center Administrator governs the whole layer: spell-check mode (off, suggest only, auto-replace and suggest), completion enablement globally or per team and per user, enabled languages, context-source toggles for completion, the admin-managed custom dictionary, and per-tenant budget caps on completion inference. Admin settings override agent preferences, satisfying enforceability; agent-level choice exists only where the admin allows it. Telemetry captures suggestion acceptance rate, auto-correct revert rate, and handling-time deltas per agent and service — the tuning signal for thresholds and the evidence base for GOSI's stated AHT and consistency objectives.

4 Integration with Bright Pattern

Agent Desktop 2.0

The layer ships inside Agent Desktop as part of the chat and email working areas — no extension, no separate window. At login, the desktop pulls the tenant's composer-assistance configuration alongside its existing bootstrap, warms the spelling worker, and registers the completion trigger only for agents whose privileges allow it. The email editor's rich-text model is the harder integration (formatting can split words across text nodes), which is why the token-to-range bookkeeping lives in the shared integration layer rather than per channel.

Contact Center Administrator

Configuration follows existing platform patterns. Enablement rides on the privilege system — a new `Use composer assistance` privilege, with a separate `Use inline completion` privilege for the model-driven half, mirroring how `Use Real-time Intent-based Assist` already gates Agent Assist — so per-team rollout uses the same role and team machinery administrators already know. The custom dictionary and mode settings live at tenant scope with optional per-service overrides, consistent with how service-level properties such as transcription settings are handled today.

Intelligence Engine

Tier 2 is two new capabilities on existing plumbing. The completion endpoint is a new low-latency inference path with per-tenant rate and budget controls, but its context assembly reuses infrastructure Real-Time Suggestions already established: the live transcript pipeline, intent and embedding retrieval against tenant content, and knowledge base access. PII handling inherits the Intelligence Engine's existing tenant-boundary and retention rules; completion request logs are masked or short-retention since prompts necessarily contain customer conversation text. Suggestion telemetry lands in the same reporting layer that serves interaction analytics, so checklist-style adherence dashboards can be extended to completion acceptance without new reporting infrastructure.

Coexistence with existing assistance features

The layer is deliberately positioned below the existing assistance stack. Real-Time Suggestions answers *what to say* at the level of knowledge — surfacing articles and guidance when intents fire. Canned responses provide *approved full messages*. The composer assistance layer works at the level of *the words currently being typed* — fixing them, and finishing them in a voice trained toward the canned library. Because completion is grounded in canned responses and KB content, heavier use of the composer layer reinforces rather than bypasses the curated content, and no arbitration between features is needed: they operate on different objects.

5 Delivery phasing

Phase	Scope	Serves
1	Spell checking, English and Arabic, suggest-only mode, behind a tenant feature flag with GOSI as beta tenant	#60254
2	Auto-replace with confidence layer, admin custom dictionary, mode enforcement	#60254

3	Inline completion grounded in transcript and canned responses; per-team privilege rollout; telemetry dashboard	#60145
4	Knowledge base grounding for completion; on-demand grammar polish action	#60145 + #60254

The ordering matches the customer's stated priorities — spelling before grammar, generic completion before KB-grounded completion — and puts the cheapest, lowest-risk value (Phase 1) in agents' hands earliest while the model-tier cost controls are proven on a limited population.

6 Open items and risks

Four items need resolution before the architecture is committed. First, Arabic dictionary licensing: AyaSpell's license family must be confirmed by legal; the fallback paths (commercial dictionary, clean-room wordlist via CAMeL Tools) carry cost and schedule respectively. Second, Arabic quality assurance: auto-replace thresholds and normalization rules need a native-speaker test pass, because a visibly wrong Arabic auto-correction will get the feature disabled tenant-wide faster than any metric can justify it. Third, completion economics: inline completion generates far more inference volume than any current Intelligence Engine feature, so the budget-cap and caching design should be validated against GOSI's projected agent count before pricing and packaging are set. Fourth, a question back to the customer: should auto-corrected and completion-accepted text be flagged in the interaction record for quality management, given transcripts will store the final corrected output.